# Message queues & async jobs

11 min read  Updated 24 Jul 2026  Topic 70 of 70



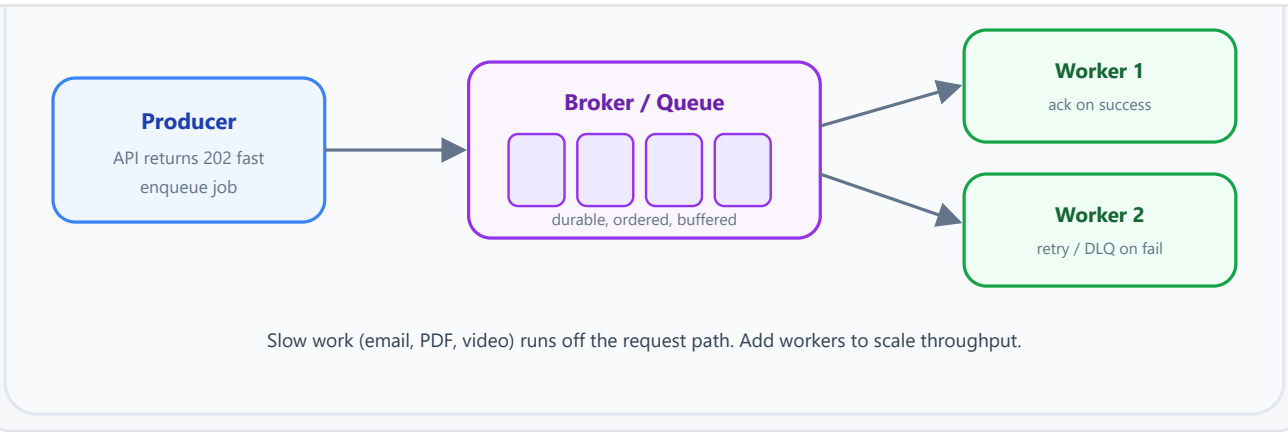
## Why it matters

Some work does not belong on the request path. When a user signs up, they should not wait for a welcome email to send, a thumbnail to render, or an analytics event to reach a warehouse. If you do that work inline, the request is as slow as your slowest dependency, and a hiccup in the email provider becomes a failed signup. Worse, a traffic spike hits your database and every third-party API at once, with nothing to smooth the load.

A **message queue** is the tool that fixes all of this. It lets a service hand off a unit of work and respond immediately, while separate workers pick the work up and do it reliably in the background — retrying on failure, absorbing spikes, and scaling independently. Queues are the backbone of resilient backends and a favourite interview topic because they force you to reason about delivery guarantees, ordering, idempotency, and failure — the parts of distributed systems that separate people who have run things in production from those who have only read about them.

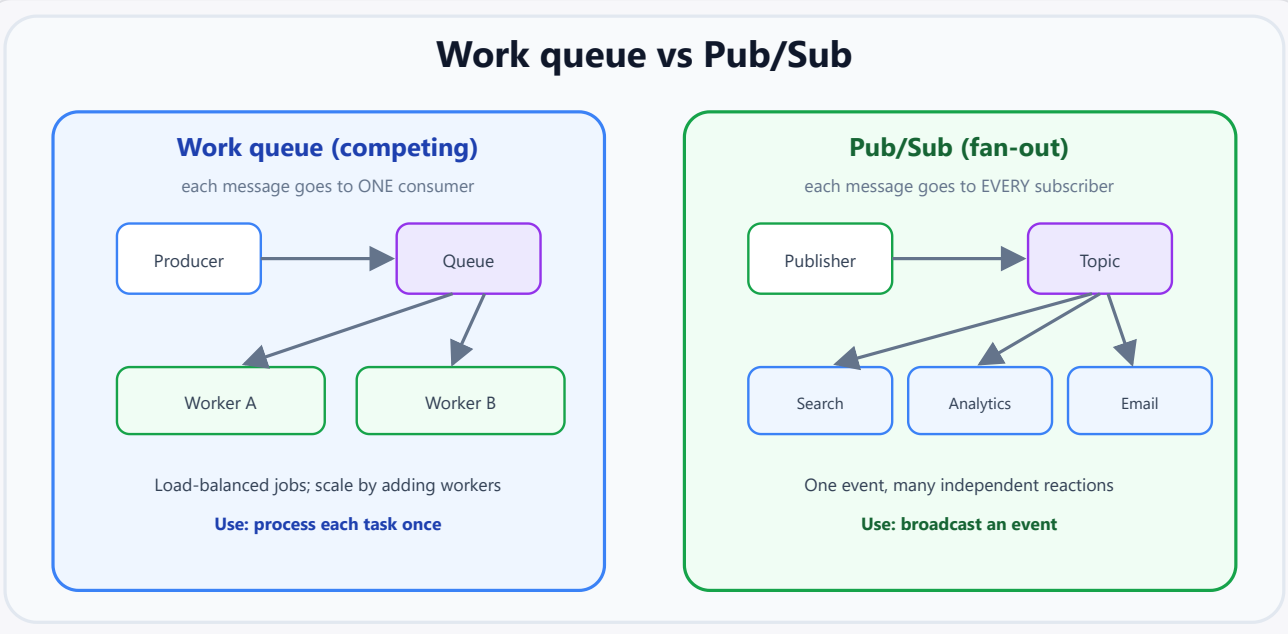
## The concept, simply

A queue sits between a **producer** (who creates work) and one or more **consumers**/workers (who do it). The producer *enqueues* a message and moves on; the broker stores it durably; a worker *dequeues* it, does the job, and **acknowledges** success. If the worker crashes before acking, the broker redelivers the message to someone else — that is the reliability guarantee at the heart of the pattern.




The producer responds instantly after enqueuing; durable, buffered messages let workers do slow work in the background and scale out horizontally by simply adding more consumers.

There are two fundamental delivery shapes, and mixing them up is a classic mistake. A **work queue** load-balances: each message is handled by exactly *one* of the competing workers — perfect for "do this job once". **Pub/Sub** fans out: each message is delivered to *every* subscriber — perfect for "this event happened, let anyone interested react". Sending payment jobs via pub/sub would charge the card three times; broadcasting an event through a single work queue would let only one of your three interested services hear it.




Work queue = one message to one consumer (load-balanced jobs); Pub/Sub = one message to every subscriber (broadcast an event) — choosing the wrong one either drops work or duplicates it.

## How it works (deep)

The behaviour you must reason about is the **delivery guarantee**. *At-most-once* may drop messages (fire and forget). *Exactly-once* is the fantasy everyone wants and almost nothing truly delivers end-to-end. In practice, real brokers give you **at-least-once**: a message is delivered until it is acknowledged, so if a worker dies mid-job or its ack is lost, the message comes back — which means **duplicates are guaranteed to happen eventually**. The engineering consequence is non-negotiable: **consumers must be idempotent**. Processing the same "charge order 42" message twice must charge once. You achieve this by deduping on a stable id, using upserts, or recording processed message ids.

The **acknowledgement** is the other load-bearing idea. You ack *after* the work succeeds, never before. Ack too early and a crash loses the job silently; the whole reliability story depends on the broker holding the message until you confirm it is done. Messages that keep failing must not loop forever — after N attempts they go to a **dead-letter queue (DLQ)**, a side channel where poison messages wait for a human instead of blocking the pipeline or burning CPU on infinite retries.

The major tools embody different philosophies. **RabbitMQ** is a classic *smart broker*: producers publish to an *exchange* which routes to queues by binding rules (direct, topic, fanout); the broker tracks per-message acks and delivery, and a consumed message is gone. It excels at flexible routing and per-message work distribution. **Kafka** is a different animal — a *distributed, append-only log*. Messages are appended to partitioned *topics* and **retained** (for days, or forever) regardless of consumption; each consumer group tracks its own *offset* (its read position). That makes Kafka superb for high-throughput event streaming, replaying history, and multiple independent consumers reading the same stream — but it is not a task queue with per-message acks and selective routing. Ordering in Kafka is guaranteed only *within a partition*, which is why the partition *key* (e.g. userId) matters. **BullMQ** (Redis-backed) is the pragmatic Node default for *application job queues*: retries, delays, scheduled/repeatable jobs, rate limiting, and priorities, with almost no ops overhead if you already run Redis.

The rule of thumb: reach for a **job queue** (BullMQ) when you have discrete background *tasks* to run reliably; reach for a **log** (Kafka) when you have a high-volume *stream of events* many systems consume and you value replay; reach for a **broker** (RabbitMQ) when you need rich routing and per-message work distribution between services. And

## Code examples

A BullMQ producer and worker — the everyday Node pattern. The API enqueues and returns; the worker processes with automatic retries and backoff.

JAVASCRIPT Copy

```
import { Queue, Worker } from 'bullmq';

const connection = { host: '127.0.0.1', port: 6379 };
const emailQueue = new Queue('emails', { connection });

// Producer: inside your signup handler, respond immediately after this
await emailQueue.add(
  'welcome',
  { userId: '42', to: 'ada@example.com' },
  {
    jobId: 'welcome-42',           // stable id → natural de-dupe
    attempts: 5,                   // retry on failure
    backoff: { type: 'exponential', delay: 1000 },
    removeOnComplete: 1000,
  },
);

// Worker: a separate process/file
new Worker('emails', async (job) => {
  if (job.name === 'welcome') {
    await sendEmail(job.data.to); // if this throws, BullMQ retries
  }
}, { connection, concurrency: 10 });

async function sendEmail(to) { /* ... */ }
```

An idempotent consumer — because at-least-once means this *will* run twice for some message. Record the message id and no-op on repeats.

JAVASCRIPT Copy

```
  // Atomically claim this message id; if already processed, skip.
  const firstTime = await db.tryInsertProcessed(job.id); // false if it ex
  if (!firstTime) return; // duplicate delivery → do nothing

  await paymentProvider.charge(orderId); // safe to run exactly once now
}

// db.tryInsertProcessed: INSERT ... ON CONFLICT DO NOTHING; returns rowCo
```

RabbitMQ work queue with manual ack and a dead-letter queue — the two features that make delivery reliable.

JAVASCRIPT

Copy

```
import amqplib from 'amqplib';

const conn = await amqplib.connect(process.env.AMQP_URL);
const ch = await conn.createChannel();

// Main queue routes failures to a DLQ after they are rejected
await ch.assertQueue('jobs', {
  durable: true,
  deadLetterExchange: '',
  deadLetterRoutingKey: 'jobs.dead',
});
await ch.assertQueue('jobs.dead', { durable: true });

ch.prefetch(20); // do not fetch a message until we free capacity

ch.consume('jobs', async (msg) => {
  try {
    await doWork(JSON.parse(msg.content.toString()));
    ch.ack(msg);                      // success → remove from queue
  } catch (err) {
    ch.nack(msg, false, false);       // fail, do not requeue → goes to D
  }
});
```

A Kafka consumer group with manual offset commit — note offsets and retention instead of per-message acks.

JAVASCRIPT Copy

```
import { Kafka } from 'kafkajs';

const kafka = new Kafka({ clientId: 'orders', brokers: ['localhost:9092'] });
const consumer = kafka.consumer({ groupId: 'analytics' }); // group tracks

await consumer.connect();
await consumer.subscribe({ topic: 'order-events', fromBeginning: false });

await consumer.run({
  autoCommit: false, // commit only after successful processing
  eachMessage: async ({ topic, partition, message }) => {
    const event = JSON.parse(message.value.toString());
    await project(event);              // update a read model
    await consumer.commitOffsets([{  // advance our read position
      topic, partition, offset: (Number(message.offset) + 1).toString(),
    }]);
  },
});

async function project(_e) {}
```

## Common mistakes & gotchas

- **Non-idempotent consumers.** At-least-once delivery means duplicates are inevitable. If reprocessing a message double-charges, double-emails, or double-counts, the design is broken — dedupe on a stable id.
- **Acking before the work is done.** Ack after success only. Early acks turn "worker crashed mid-job" into silent data loss, defeating the entire point of a queue.

- **No dead-letter queue.** A single poison message with no DLQ retries forever, burns CPU, and can wedge the whole queue. Cap attempts and route failures aside for inspection.
- **Confusing work queues with pub/sub.** Using pub/sub for "run this job once" duplicates the work across subscribers; using one work queue for "notify everyone interested" means only one consumer hears it.
- **Assuming global ordering.** Concurrent workers process out of order by design, and Kafka only orders *within a partition*. If order matters, key related messages to the same partition/consumer and keep concurrency at 1 for that key.
- **Unbounded prefetch.** Letting a worker grab thousands of messages at once starves other workers and blows up memory; set a sane prefetch/concurrency so work is distributed.
- **Putting huge payloads on the queue.** Messages should be small pointers ("process file X at s3://..."), not multi-megabyte blobs. Store the payload elsewhere and pass a reference.
- **Treating Kafka like a task queue (or RabbitMQ like a log).** Kafka retains and replays a stream but is not built for selective per-message acking; RabbitMQ distributes tasks but is not a durable replayable log. Wrong tool, endless friction.

## Best practices

- **Make every consumer idempotent** using a stable message/operation id (dedupe table, upsert, or conditional insert) — assume every message can arrive more than once.
- **Ack only after success,** and configure bounded retries with exponential backoff plus a dead-letter queue for the messages that still fail.
- **Keep messages small and self-describing:** include a type, a version, and an id; put large data behind a reference and pass the pointer.
- **Right-size concurrency and prefetch** so load spreads across workers and a slow job cannot hoard the queue; scale throughput by adding consumers.
- **Pick the delivery shape deliberately:** work queue for "process once", pub/sub for "broadcast an event" — and where order matters, partition by a key.

- **Match the tool to the job:** BullMQ for app job queues, Kafka for high-throughput replayable streams, RabbitMQ for rich routing — do not force one to be the others.
- **Monitor depth, age, and DLQ size.** A growing backlog or an oldest-message age that keeps climbing is the earliest signal that consumers cannot keep up.
- **Only add a queue when inline work actually hurts;** it is real infrastructure with its own failure modes, not a free abstraction.

## ⭐ Interview lens

**Q: Why put a message queue in front of background work?** To decouple producers from consumers: the request responds immediately, slow/failable work runs out-of-band with retries, traffic spikes are buffered instead of hammering downstreams, and you scale workers independently of the API. It converts "as slow and fragile as my slowest dependency" into "enqueue and move on".

**Q: What delivery guarantee do real queues give, and what does that force on you?** At-least-once in practice — a message is redelivered until acknowledged, so duplicates will happen. That forces *idempotent consumers*: dedupe on a stable id so reprocessing is a no-op. *Trap:* claiming "exactly-once" — true end-to-end exactly-once is effectively unattainable; you approximate it with at-least-once delivery plus idempotency.

**Q: Work queue vs pub/sub — what is the difference?** A work queue delivers each message to exactly one of the competing consumers (load-balanced, "do once"); pub/sub delivers each message to every subscriber (fan-out, "broadcast"). Choosing wrong either drops work or duplicates it across services.

**Q: RabbitMQ vs Kafka — when each?** RabbitMQ is a smart broker with flexible routing and per-message acks; messages are consumed and gone — great for task distribution between services. Kafka is a partitioned, retained, append-only log

streaming and replayable history → Kafka.

**Q: A message keeps failing. What happens to it?** With bounded retries it is attempted N times (ideally with backoff); after that it is routed to a dead-letter queue for inspection rather than requeued forever. Without a DLQ a poison message loops endlessly and can stall the pipeline.

**Q: How do you preserve ordering when it matters?** Accept that concurrent workers reorder by default. Route all messages that must stay ordered to the same partition/queue via a consistent key (e.g. userId) and process that key with a single consumer/concurrency of 1. Kafka guarantees order only within a partition, so the partition key is the lever.

## Cheat-card

- **Queue =** producer → durable broker → worker; respond fast, process slow work off the request path.
- **Delivery:** real brokers = **at-least-once** → duplicates happen → consumers MUST be idempotent (dedupe on id).
- **Ack after success only;** failed messages → bounded retries + backoff → dead-letter queue.
- **Shape:** work queue = one message to one consumer; pub/sub = one message to all subscribers.
- **Tools:** BullMQ (Redis) = app job queues; Kafka = retained/replayable log, order per partition; RabbitMQ = routing + per-message acks.
- **Ordering:** not global; key related messages to one partition/consumer if order matters.
- **Hygiene:** small referenced payloads, tuned prefetch/concurrency, monitor depth/age/DLQ; add a queue only when inline hurts.

1. What delivery guarantee do real message brokers give in practice, and what does it force on consumers?

- ◯ Exactly-once delivery, so consumers can assume each message runs a single time
- ◯ At-least-once delivery — messages are redelivered until acknowledged, so duplicates will happen and consumers must be idempotent (dedupe on a stable id)
- ◯ At-most-once delivery, so consumers must retry dropped messages themselves
- ◯ Ordered-once delivery, so consumers can rely on strict global ordering

Check answer

2. What happens to a message that keeps failing, and why is that mechanism important?

- ◯ It is acknowledged immediately so it stops consuming resources
- ◯ It is silently discarded after the first failure
- ◯ After N bounded retries it is routed to a dead-letter queue (DLQ) for inspection, instead of looping forever and wedging the pipeline
- ◯ It is broadcast to every subscriber until one succeeds

Check answer

3. What is the difference between a work queue and pub/sub, and what goes wrong if you swap them?

- ◯ A work queue broadcasts to all consumers; pub/sub delivers to one — swapping them changes only latency
- ◯ They are identical; the names are interchangeable

A work queue guarantees ordering; pub/sub guarantees delivery — neither affects duplication

- A work queue delivers each message to exactly one competing consumer (do once); pub/sub delivers each message to every subscriber (broadcast) — swapping them either drops work or duplicates it

Check answer

Previous
Microservices in Node

## LEARN MORE

### ARTICLES & DOCS

- RabbitMQ Tutorial - "Hello World!" (JavaScript/amqplib)
  rabbitmq.com
- RabbitMQ Tutorial - Work Queues (JavaScript/amqplib)
  rabbitmq.com

### VIDEOS




Kafka vs RabbitMQ: The Best Message Queue Explained
The Coding Gopher

Hand-picked references for this lesson.